AI / COMPUTER VISION / INTERACTIVE WEB

DigitLens

CNN-Based Handwritten Digit Recognition and Streamlit Application

DATASET	MODEL	DELIVERABLES
MNIST: digits 0-9	**PyTorch CNN**	**1 Python + 1 PDF**

Student: Varun D. Sawalkar

Batch: September Batch

Date: 07 November 2026

App topic: Interactive handwritten-digit image classifier

Project overview

A small convolutional neural network learns spatial patterns in 28 x 28 grayscale images, predicts one of ten digit classes, and powers a Streamlit dashboard for user-uploaded images. The application also includes training and evaluation views.

Execution status: Code passed synthetic offline checks. MNIST training, results, and live app screenshot must be captured on a machine with dataset access before final submission.

01 Project objective and dataset

Part A: Classify digit images using a trained CNN, quantify generalization on held-out images, and explain prediction successes and failures.

Part B: Provide a usable Streamlit interface with clear instructions, an image upload or sample selector, prediction processing, visible outputs, and error handling.

Dataset specification

Property	Value
Source / classes	MNIST, handwritten numerals 0 to 9
Image dimensions	28 x 28 pixels; one grayscale channel
Original split	60,000 training images; 10,000 held-out test images
Project train / validation	54,000 train; 6,000 validation (seed = 42)

Preprocessing workflow

Image -> grayscale	**Center / crop digit**	**Resize to 28 x 28**	**Normalize pixels**	**CNN prediction**

MNIST image examples (digits 0-9)

Image evidence is pending

Run the MNIST training command to generate genuine digit examples. Then use --export-report again.

The original MNIST pixels are converted to tensors in [0, 1] and normalized with mean 0.1307 and standard deviation 0.3081. User uploads additionally handle background polarity, centering, and blank-image rejection.

02 Model architecture and training

CNN design

Stage	Implementation
Input	Grayscale image [1, 28, 28]
Convolution block 1	Conv2d(1, 16, kernel=3, padding=1) + ReLU + MaxPool2d(2)
Convolution block 2	Conv2d(16, 32, kernel=3, padding=1) + ReLU + MaxPool2d(2)
Classifier	Flatten 1,568 units -> Dense 64 -> ReLU -> Dropout(0.25)
Output	Dense 10 logits, predicted class = argmax(logits)

Trainable parameters: 105,866 (computed from the defined PyTorch network).

Training protocol

Setting	Choice
Loss	Categorical cross-entropy over raw class logits
Optimizer	Adam, learning rate 0.001
Validation	Fixed 54k/6k split; keep checkpoint with lowest validation loss
Test	Evaluate once after model selection on separate 10k examples
Reproducibility	Seed = 42; batch size = 128; default = 3 epochs

How the CNN learns

Convolution filters detect local edges and strokes; pooling reduces spatial dimensions; the dense layers combine these features into ten digit scores. Backpropagation updates filter weights to minimize loss.

For the required model-summary screenshot: open the Streamlit 'Train & setup' tab, where the full architecture is displayed.

03 CNN evaluation and findings

TEST ACCURACY	TEST LOSS	CORRECT / 10,000
Not yet measured	**Not yet measured**	**Not yet measured**

Training and validation curves

Training / validation evidence

Generated after real training: epoch accuracy and cross-entropy loss for training and validation splits.

Test errors and correct predictions

Confusion matrix pending

10 x 10 matrix created from 10,000 test predictions.

Predictions pending

8 correct and 8 incorrect genuine MNIST predictions.

Observation pending: Report actual test accuracy and which digit pairs the confusion matrix shows are most often mixed up. Do not replace this with an estimated score.

04 Streamlit application

DigitLens is a single-page, three-tab web dashboard. Users can upload a photograph or test a real MNIST example, compare the original image with its 28 x 28 input, and inspect ten class scores.

Live application screenshot

Actual Streamlit browser screenshot required

Run the web app, predict a digit, capture the browser screen, save as digitlens_runtime/app_screenshot.png, then regenerate this report.

Features and application flow

Tab / capability	User-visible output
Predict digit	Upload PNG/JPEG or use a MNIST test sample; show image, class and scores
Model results	Display saved metrics, training curves, example predictions and confusion matrix
Train & setup	CNN summary, instructions, configurable epochs and in-app training button
Input handling	Reject empty/invalid/oversized image; handle untrained or missing model

The interface explains model uncertainty, explicitly distinguishes softmax confidence from a guarantee, and warns that uploaded photos can differ from MNIST's clean training distribution.

05 Conclusion and reproducibility

Results summary

The model, optimizer, image preprocessing and empty-input handling have passed an offline synthetic smoke test. The genuine MNIST train/test experiment cannot be certified until the real dataset has been downloaded and training, metrics and screenshots have been recorded.

Challenges and improvements

Potential challenges: ambiguity in handwritten strokes, out-of-distribution phone photos, white/black background variation, and computing time. Some digit pairs may be visually similar.

Improvements: collect real user handwriting, use controlled rotation/translation augmentation on training data only, add a confidence-based retake prompt, and calibrate probabilities.

How to run this project

```
python -m pip install torch torchvision streamlit numpy pillow matplotlib reportlab
python Varun_CNN_Streamlit_Project.py --smoke-test
python Varun_CNN_Streamlit_Project.py --train --epochs 3
python -m streamlit run Varun_CNN_Streamlit_Project.py
# Save browser screenshot: digitlens_runtime/app_screenshot.png
python Varun_CNN_Streamlit_Project.py --export-report
```

Submission checklist

[ ] One Python file with both parts [ ] MNIST test metrics [ ] Accuracy/loss plots [ ] Correct and incorrect samples [ ] Actual Streamlit screenshot [ ] PDF 4-6 pages

References

1. LeCun et al., MNIST database of handwritten digits: https://yann.lecun.com/exdb/mnist/
2. PyTorch and TorchVision documentation: https://pytorch.org/vision/
3. Streamlit documentation: https://docs.streamlit.io/

Integrity note: Illustrative/test-only checks are clearly distinguished from measured MNIST results. This report is regenerated from real JSON metrics and saved plots after execution.